

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ,
МЕХАНИКИ И ОПТИКИ»**

Факультет безопасности информационных технологий

**Дисциплина:
«Информационная безопасность баз данных»**

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

Разграничение доступа

Выполнил:

Студент гр. N3248

Жук Анастасия Валерьевна



Проверил:

Салихов Максим Русланович

Санкт-Петербург

2025 г.

Цель работы: изучение и практическое освоение механизмов разграничения доступа в СУБД PostgreSQL, включая управление правами пользователей, использование подсхем, представлений, безопасности на уровне строк (RLS) и аудита изменений данных с помощью триггеров.

Задачи:

1. Научиться настраивать базовые права доступа.
2. Научиться работать с подсхемами.
3. Рассмотреть управление доступом через представления.
4. Реализовать безопасность на уровне строк (RLS).
5. Реализовать аудит изменений.
6. Протестировать полученные ограничения.

Ход работы

1. Подготовить таблицы для выполнения перечисленных ниже задач.
Достаточно 2-3 таблиц для п. 1-5 ниже.

Воспользуемся ранее созданными таблицами Authors и Books:

```
CREATE TABLE Authors (  
  author_id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE Books (  
  book_id SERIAL PRIMARY KEY,  
  title VARCHAR(100) NOT NULL,  
  author_id INTEGER,  
  FOREIGN KEY (author_id) REFERENCES Authors(author_id)  
);
```

```
postgres=# select * from authors;  
Active code page: 65001  
author_id |      name  
-----+-----  
1 | Лев Толстой  
2 | Фёдор Достоевский  
3 | Антон Чехов  
4 | Михаил Булгаков  
5 | Александр Пушкин  
6 | Михаил Лермонтов  
(6 строк)
```



```
postgres=# select * from books;  
Active code page: 65001  
book_id |      title      | author_id  
-----+-----+-----  
1 | Война и мир      | 1  
2 | Белые ночи       | 2  
3 | Пиковая дама     | 5  
4 | Кавказский пленник | 1  
5 | Преступление и наказание | 2  
6 | Бедные люди      | 2  
7 | Вишневый сад     | 3  
8 | Толстый и тонкий | 3  
9 | Униженные и оскорбленные | 2  
10 | Роковые яйца     | 4  
11 | Хамелеон         | 3  
12 | Мастер и Маргарита | 4  
13 | Евгений Онегин   | 5  
14 | Анна Каренина    | 1  
15 | Братья Карамазовы | 2  
16 | Дама с собачкой  | 3  
17 | Собачье сердце   | 4  
18 | Капитанская дочка | 5  
19 | Каштанка         | 3  
20 | Руслан и Людмила | 5  
21 | Пари             | 3  
22 | Крыжовник        | 3  
24 | Дубровский       | 5  
25 | Мцыри            | 6  
(24 строки)
```

2. Выдать права 3 пользователям. Пользователь User1 должен иметь полный доступ к таблице. User2 должен иметь право на вставку, select-запросы и обновление значений в таблицах. User3 должен иметь право на удаление строк из таблиц, а также возможность делегировать свои права любому пользователю.

Создание пользователей:

```
CREATE USER User1 WITH PASSWORD 'password1';  
CREATE USER User2 WITH PASSWORD 'password2';  
CREATE USER User3 WITH PASSWORD 'password3';
```

```
postgres=# CREATE USER User1 WITH PASSWORD 'password1';  
CREATE ROLE  
postgres=# CREATE USER User2 WITH PASSWORD 'password2';  
CREATE ROLE  
postgres=# CREATE USER User3 WITH PASSWORD 'password3';  
CREATE ROLE
```

Права для User1 (полный доступ к таблице Authors):

```
GRANT ALL PRIVILEGES ON TABLE Authors TO User1;
```

(к нескольким таблицам):

```
GRANT ALL PRIVILEGES ON TABLE Authors, Books TO User1;
```

Права для User2 (INSERT, SELECT, UPDATE):

```
GRANT INSERT, SELECT, UPDATE ON TABLE Authors, Books TO  
User2;
```

Права для User3 (DELETE с возможностью делегирования):

```
GRANT DELETE ON TABLE Authors, Books TO User3 WITH GRANT  
OPTION;
```

```
postgres=# GRANT ALL PRIVILEGES ON TABLE Authors TO User1;  
GRANT  
postgres=# GRANT INSERT, SELECT, UPDATE ON TABLE Authors, Books TO User2;  
GRANT  
postgres=# GRANT DELETE ON TABLE Authors, Books TO User3 WITH GRANT OPTION;  
GRANT
```

3. Предоставить право на удаление от пользователя User3 пользователю User4 и проверить все выданные права.

Создание User4:

```
CREATE USER User4 WITH PASSWORD 'password4';
```

Переключение на User3 и делегирование прав:

```
SET ROLE User3;  
GRANT DELETE ON TABLE Authors, Books TO User4;  
RESET ROLE;
```

SET ROLE User3 переключает текущую сессию на выполнение от имени пользователя User3 (с его правами).

RESET ROLE возвращает сессию к первоначальному пользователю (обычно это пользователь, который подключился к БД, например, postgres).

```
postgres=# CREATE USER User4 WITH PASSWORD 'password4';
CREATE ROLE
postgres=# SET ROLE User3;
SET
postgres=> GRANT DELETE ON TABLE Authors, Books TO User4;
GRANT
postgres=> RESET ROLE;
RESET
postgres=# \dp Authors
Active code page: 65001
```

Схема	Имя	Тип	Права доступа	Права для столбцов	Политики
public	authors	таблица	postgres=arwdDxtm/postgres+ user1=arwdDxtm/postgres + user2=arw/postgres + user3=d*/postgres + user4=d/user3		

(1 строка)

```
postgres=# \dp Books
Active code page: 65001
```

Схема	Имя	Тип	Права доступа	Права для столбцов	Политики
public	books	таблица	postgres=arwdDxtm/postgres+ user2=arw/postgres + user3=d*/postgres + user4=d/user3		

(1 строка)

4. Отменить все предоставленные выше права.

```
REVOKE ALL PRIVILEGES ON TABLE Authors FROM user1;
REVOKE ALL PRIVILEGES ON TABLE Authors, Books FROM user2;
REVOKE ALL PRIVILEGES ON TABLE Authors, Books FROM user3
CASCADE;
REVOKE ALL PRIVILEGES ON TABLE Authors, Books FROM user4;
```

```
postgres=# REVOKE ALL PRIVILEGES ON TABLE Authors FROM user1;
REVOKE
postgres=# REVOKE ALL PRIVILEGES ON TABLE Authors, Books FROM user2;
REVOKE
postgres=# REVOKE ALL PRIVILEGES ON TABLE Authors, Books FROM user3 CASCADE;
REVOKE
postgres=# REVOKE ALL PRIVILEGES ON TABLE Authors, Books FROM user4;
REVOKE
```

CASCADE позволяет удалить права со всеми случаями делегирования.

5. Создать подсхему авторизации для User1 и User2 с различным набором таблиц (служебное слово AUTHORIZATION у команды CREATE SCHEMA).

Подсхема для User1:

```
CREATE SCHEMA user1_schema AUTHORIZATION user1;
```

Подсхема для User2:

```
CREATE SCHEMA user2_schema AUTHORIZATION user2;
```

Создаем таблицы в подсхемах:

```
CREATE TABLE user1_schema.table1 (  
    id SERIAL PRIMARY KEY,  
    info TEXT  
);
```

```
CREATE TABLE user1_schema.table2 (  
    id SERIAL PRIMARY KEY,  
    details TEXT  
);
```

```
CREATE TABLE user2_schema.table1 (  
    id SERIAL PRIMARY KEY,  
    info TEXT  
);
```

```
postgres=# CREATE SCHEMA user1_schema AUTHORIZATION user1;  
CREATE SCHEMA  
postgres=# CREATE SCHEMA user2_schema AUTHORIZATION user2;  
CREATE SCHEMA  
postgres=# CREATE TABLE user1_schema.table1 (  
postgres(#      id SERIAL PRIMARY KEY,  
postgres(#      info TEXT  
postgres(# );  
CREATE TABLE  
postgres=# CREATE TABLE user1_schema.table2 (  
postgres(#      id SERIAL PRIMARY KEY,  
postgres(#      details TEXT  
postgres(# );  
CREATE TABLE  
postgres=# CREATE TABLE user2_schema.table1 (  
postgres(#      id SERIAL PRIMARY KEY,  
postgres(#      info TEXT  
postgres(# );  
CREATE TABLE
```

```

postgres=# \dn
Active code page: 65001
      Список схем
  Имя      | Владелец
-----+-----
 public    | pg_database_owner
 user1_schema | user1
 user2_schema | user2
(3 строки)

postgres=# \dt user1_schema.*
Active code page: 65001
      Список отношений
  Схема      | Имя      | Тип      | Владелец
-----+-----+-----+-----
 user1_schema | table1   | таблица  | postgres
 user1_schema | table2   | таблица  | postgres
(2 строки)

postgres=# \dt user2_schema.*
Active code page: 65001
      Список отношений
  Схема      | Имя      | Тип      | Владелец
-----+-----+-----+-----
 user2_schema | table1   | таблица  | postgres
(1 строка)

```

Передать владение таблицами соответствующим пользователям:

```

ALTER TABLE user1_schema.table1 OWNER TO user1;
ALTER TABLE user1_schema.table2 OWNER TO user1;
ALTER TABLE user2_schema.table1 OWNER TO user2;

```

```

postgres=# ALTER TABLE user1_schema.table1 OWNER TO user1;
ALTER TABLE
postgres=# ALTER TABLE user1_schema.table2 OWNER TO user1;
ALTER TABLE
postgres=# ALTER TABLE user2_schema.table1 OWNER TO user2;
ALTER TABLE
postgres=# \dn
Active code page: 65001
      Список схем
  Имя      | Владелец
-----+-----
 public    | pg_database_owner
 user1_schema | user1
 user2_schema | user2
(3 строки)

postgres=# \dt user1_schema.*
Active code page: 65001
      Список отношений
  Схема      | Имя      | Тип      | Владелец
-----+-----+-----+-----
 user1_schema | table1   | таблица  | user1
 user1_schema | table2   | таблица  | user1
(2 строки)

postgres=# \dt user2_schema.*
Active code page: 65001
      Список отношений
  Схема      | Имя      | Тип      | Владелец
-----+-----+-----+-----
 user2_schema | table1   | таблица  | user2
(1 строка)

```

Передача владения таблицами соответствующим пользователям позволяет дать пользователям полный контроль над своими таблицами и обеспечивает реальную автономность подсистем.

6. Создать представление как объединенный набор столбцов из разных таблиц. С помощью команды grant ограничить доступ к представлению.

Цель представления — показать данные из обеих таблиц.

Создание представления:

```
CREATE VIEW combined_view AS
SELECT
    u1.id as user1_id,
    u1.info as user1_info,
    u2.id as user2_id,
    u2.info as user2_info
FROM
    user1_schema.table1 u1
FULL OUTER JOIN
    user2_schema.table1 u2 ON u1.id = u2.id;
```

Ограничение доступа к представлению:

```
GRANT SELECT ON combined_view TO user1;
```

Проверяем доступ:

```
SET ROLE user1;
SELECT * FROM combined_view;
RESET ROLE;
```

```
SET ROLE user2;
SELECT * FROM combined_view;
RESET ROLE;
```



```

postgres=# select * from combined_view;
Active code page: 65001
 user1_id | user1_info | user2_id | user2_info
-----+-----+-----+-----
(0 строк)

postgres=# SET ROLE user1;
SET
postgres=> SELECT * FROM combined_view;
Active code page: 65001
 user1_id | user1_info | user2_id | user2_info
-----+-----+-----+-----
(0 строк)

postgres=> RESET ROLE;
RESET
postgres=# SET ROLE user2;
SET
postgres=> SELECT * FROM combined_view;
ОШИБКА: нет доступа к представлению combined_view
postgres=> RESET ROLE;
RESET

```

7. Настроить безопасность на уровне строк (RLS), политика должна быть создана на основе текущего пользователя, и протестировать ее.

Добавляем столбец для хранения владельца записи:

```
ALTER TABLE Books ADD COLUMN owner VARCHAR(50) DEFAULT CURRENT_USER;
```

Включаем RLS для таблицы Books:

```
ALTER TABLE Books ENABLE ROW LEVEL SECURITY;
```

Создаем политику: пользователь видит только свои книги:

```
CREATE POLICY books_owner_policy ON Books
FOR ALL TO PUBLIC
USING (owner = CURRENT_USER);
```

```

postgres=# ALTER TABLE Books ADD COLUMN owner VARCHAR(50) DEFAULT CURRENT_USER;
ALTER TABLE
postgres=# ALTER TABLE Books ENABLE ROW LEVEL SECURITY;
ALTER TABLE
postgres=# CREATE POLICY books_owner_policy ON Books
postgres=# FOR ALL TO PUBLIC
postgres=# USING (owner = CURRENT_USER);
CREATE POLICY

```

Тестирование:

Для того, чтобы осуществить проверку ограничений, нужно дать доступ пользователям к таблицам:

```
GRANT ALL PRIVILEGES ON TABLE Authors, Books TO User1;
GRANT ALL PRIVILEGES ON TABLE Authors, Books TO User2;
```

```
GRANT USAGE, SELECT ON SEQUENCE books_book_id_seq TO
user1;
GRANT USAGE, SELECT ON SEQUENCE authors_author_id_seq TO
user1;
GRANT USAGE, SELECT ON SEQUENCE books_book_id_seq TO
user2;
GRANT USAGE, SELECT ON SEQUENCE authors_author_id_seq TO
user2;
```

Вставляем данные от имени user1:

```
SET ROLE user1;
INSERT INTO Books (title, author_id, owner) VALUES ('Книга User1', 1,
CURRENT_USER);
RESET ROLE;
```

Вставляем данные от имени user2:

```
SET ROLE user2;
INSERT INTO Books (title, author_id, owner) VALUES ('Книга User2', 2,
CURRENT_USER);
RESET ROLE;
```

```
postgres=# SET ROLE user1;
SET
postgres=> INSERT INTO Books (title, author_id, owner) VALUES ('Книга User1', 1, CURRENT_USER);
INSERT 0 1
postgres=> RESET ROLE;
RESET
postgres=# SET ROLE user2;
SET
postgres=> INSERT INTO Books (title, author_id, owner) VALUES ('Книга User2', 2, CURRENT_USER);
INSERT 0 1
postgres=> RESET ROLE;
RESET
```

Проверяем видимость:

```
SET ROLE user1;
SELECT * FROM Books;
RESET ROLE;
```

```
SET ROLE user2;
SELECT * FROM Books;
RESET ROLE;
```

```

postgres=# SET ROLE user1;
SET
postgres=> SELECT * FROM Books; -- Покажет только книгу User1
Active code page: 65001
 book_id | title | author_id | owner
-----+-----+-----+-----
      28 | Книга User1 |      1 | user1
(1 строка)

postgres=> RESET ROLE;
RESET
postgres=# SET ROLE user2;
SET
postgres=> SELECT * FROM Books; -- Покажет только книгу User2
Active code page: 65001
 book_id | title | author_id | owner
-----+-----+-----+-----
      29 | Книга User2 |      2 | user2
(1 строка)

postgres=> RESET ROLE;
RESET

```

Если нужно отозвать разрешения:

```

REVOKE ALL PRIVILEGES ON TABLE Authors, Books FROM User1;
REVOKE ALL PRIVILEGES ON TABLE Authors, Books FROM User2;
REVOKE USAGE, SELECT ON SEQUENCE books_book_id_seq FROM
user1;
REVOKE USAGE, SELECT ON SEQUENCE authors_author_id_seq
FROM user1;
REVOKE USAGE, SELECT ON SEQUENCE books_book_id_seq FROM
user2;
REVOKE USAGE, SELECT ON SEQUENCE authors_author_id_seq
FROM user2;

```

8. Создать триггер для регистрации вставки, обновления и удаления содержимого в определенных таблицах.

Создаем таблицу для аудита:

```

CREATE TABLE books_audit (
  audit_id SERIAL PRIMARY KEY,
  operation VARCHAR(10) NOT NULL,
  book_id INTEGER,
  changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  changed_by VARCHAR(50)
);

```

Функция триггера:

```

CREATE OR REPLACE FUNCTION log_books_changes()
RETURNS TRIGGER AS $$
BEGIN
  IF TG_OP = 'INSERT' THEN

```

```

        INSERT INTO books_audit(operation, book_id, changed_by)
        VALUES ('INSERT', NEW.book_id, current_user);
    ELSIF TG_OP = 'UPDATE' THEN
        INSERT INTO books_audit(operation, book_id, changed_by)
        VALUES ('UPDATE', NEW.book_id, current_user);
    ELSIF TG_OP = 'DELETE' THEN
        INSERT INTO books_audit(operation, book_id, changed_by)
        VALUES ('DELETE', OLD.book_id, current_user);
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

```

Создаем триггер:

```

CREATE TRIGGER books_audit_trigger
AFTER INSERT OR UPDATE OR DELETE ON Books
FOR EACH ROW EXECUTE FUNCTION log_books_changes();

```

```

postgres=# CREATE TABLE books_audit (
postgres(#      audit_id SERIAL PRIMARY KEY,
postgres(#      operation VARCHAR(10) NOT NULL,
postgres(#      book_id INTEGER,
postgres(#      changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
postgres(#      changed_by VARCHAR(50)
postgres(# );
CREATE TABLE
postgres=# CREATE OR REPLACE FUNCTION log_books_changes()
postgres=# RETURNS TRIGGER AS $$
postgres$$ BEGIN
postgres$$     IF TG_OP = 'INSERT' THEN
postgres$$         INSERT INTO books_audit(operation, book_id, changed_by)
postgres$$         VALUES ('INSERT', NEW.book_id, current_user);
postgres$$     ELSIF TG_OP = 'UPDATE' THEN
postgres$$         INSERT INTO books_audit(operation, book_id, changed_by)
postgres$$         VALUES ('UPDATE', NEW.book_id, current_user);
postgres$$     ELSIF TG_OP = 'DELETE' THEN
postgres$$         INSERT INTO books_audit(operation, book_id, changed_by)
postgres$$         VALUES ('DELETE', OLD.book_id, current_user);
postgres$$     END IF;
postgres$$     RETURN NULL;
postgres$$ END;
postgres$$ $$ LANGUAGE plpgsql;
CREATE FUNCTION
postgres=# CREATE TRIGGER books_audit_trigger
postgres=# AFTER INSERT OR UPDATE OR DELETE ON Books
postgres=# FOR EACH ROW EXECUTE FUNCTION log_books_changes();
CREATE TRIGGER

```

Помимо всех вышеописанных доступов, нужно предоставить и доступ к таблице books_audit и к последовательности для audit_id:

```

GRANT INSERT, SELECT ON books_audit TO user1, user2;

```

```
GRANT USAGE, SELECT ON SEQUENCE books_audit_audit_id_seq TO
user1, user2;
```

Вставка новой книги (от имени user1):

```
SET ROLE user1;
INSERT INTO Books (title, author_id, owner) VALUES ('Тестовая книга',
1, CURRENT_USER);
RESET ROLE;
```

Вставка новой книги (от имени user1):

```
SET ROLE user1;
INSERT INTO Books (title, author_id, owner) VALUES ('Новая книга', 7,
CURRENT_USER);
RESET ROLE;
```

Обновление книги (от имени user2 можем поменять только созданную им книгу):

```
SET ROLE user2;
UPDATE Books SET title = ' Другая новая книга' WHERE book_id = 29;
RESET ROLE;
```

Удаление книги (от имени user1 можем удалить только книгу, созданную им):

```
SET ROLE user1;
DELETE FROM Books WHERE book_id = 1;
RESET ROLE;
```

```
SELECT * FROM books_audit;
```

```
postgres=# SELECT * FROM books_audit;
```

```
Active code page: 65001
```

audit_id	operation	book_id	changed_at	changed_by
1	INSERT	34	2025-05-20 00:00:18.890564	user1
2	INSERT	35	2025-05-20 00:22:04.069423	user1
3	UPDATE	29	2025-05-20 00:29:34.958015	user2
4	DELETE	35	2025-05-20 00:31:36.792279	user1

```
(4 строки)
```

Вывод

В ходе лабораторной работы были освоены ключевые механизмы контроля доступа в PostgreSQL:

- Ролевая модель — гибкое назначение прав через GRANT/REVOKE и делегирование привилегий.
- Изоляция данных — использование подсхем и представлений для ограничения видимости данных.
- Тонкая настройка доступа — RLS позволяет фильтровать строки на уровне политик, обеспечивая безопасность даже в рамках одной таблицы.
- Аудит — триггеры позволяют отслеживать изменения данных, что критично для безопасности и отладки.

Результаты показали, что PostgreSQL предоставляет мощные инструменты для разграничения доступа, которые можно адаптировать под различные сценарии — от простого разделения прав до сложных многоуровневых политик. Для эффективного применения этих механизмов важно:

- Четко проектировать структуру прав пользователей.
- Тестировать политики доступа на реалистичных данных.
- Контролировать целостность данных через аудит.